一、替换算法总结

cache中主存数据块的替换（淘汰）算法

随机替换

基本思想：随机选取一个cache行进行替换。

优点：硬件实现容易，速度也最快。

缺点：随机替换掉的数据可能马上又要用，从而降低命中率和cache的工作效率。

- 随着cache容量的增大，随机替换的缺点所带来的负面影响会减小。
- 通过仿真试验可发现，随机替换的性能只稍逊于LFU和LRU。

先进先出FIFO算法

基本思想：将主存数据块载入cache的先后顺序，作为替换顺序。在需要进行替换时，选择最先载入主存数据块的cache行进行替换。

实现方法：记录每个cache行载入主存数据块的时刻（时间戳）。

算法特点：实现较方便（开销小），但未考虑程序的访问局部性，因为最先进入cache的主存数据块也可能是目前经常要用的，因此，可能导致cache的命中率不高。

最不经常用LFU算法

基本思想：将被命中访问次数最少的cache行淘汰。

实现方法：需要为每个cache行设置一个淘汰计数器，新载入的cache行从0开始计数，每命中访问1次，被访问cache行的淘汰计数器加1。需要进行替换时，计数值最小的cache行被替换。

算法特点：硬件成本较高（每行一个计数器）。
淘汰计数器从cache上电后就开始记录，但不能严格反映出近期的访问情况。

近期最少用LRU算法

基本思想：将近期内最少被命中访问过的cache行淘汰。

实现方法：需要为每个cache行设置一个淘汰计数器。cache行每被访问（载入/替换、命中）1次，该行的计数器被清零，其他计数值不为空的行各自的计数器被加1。需要进行替换时，计数值最大的行被替换。

算法特点：硬件成本较高（每行一个计数器）。
比较正确地反映了程序的访问局部性，因为近期最少使用的主存数据块，很可能是将来最少被访问的。

二、例题

2.1 例题一

【例题】假设cache有4行、内容初始为空、采用全相联映射方式，按以下编号序列访问相应的主存数据块：
0，1，2，1，0，2，0，3，3，4，5，6，4，5，6，4。
请给出LFU和LRU算法的访问过程，并计算两种算法的命中率。

解析



分析



颠簸（pingpong）
抖动（thrashing）

- 受到早期高频访问数据的影响，这些数据可能在初始化加载时被频繁访问，但之后很少或不再被访问。由于它们起初的访问次数很高，这些数据可能会长时间占用cache空间，导致cache污染。
- 新加入cache的数据由于访问次数较低，很容易被快速替换，尤其是当cache空间有限时。这可能导致cache末端发生“抖动”，即新数据频繁地被加载和淘汰。
- 如果数据访问模式随时间变化，LFU算法可能无法及时适应这种变化，导致一些不再频繁访问的数据仍然保留在cache中，而一些近期变得频繁访问的数据却未能及时加载到cache中。

造成“抖动”的原因?



2.2 例题二

【例题】假设cache有4行、内容初始为空、采用全相联映射方式， 按以下编号序列访问相应的主存数据块：
0，1，2，1，0，2，0，3，3，4，5，6，4，5，6，4。
请给出LFU和LRU算法的访问过程，并计算两种算法的命中率。

解析

